

```

# Fresh GNINA download from GitHub releases

import os
import subprocess

# Remove old broken GNINA file
!rm -f gnina

# GNINA release links to try
gnina_links = [
    "https://github.com/gnina/gnina/releases/download/v1.3.2/gnina",
    "https://github.com/gnina/gnina/releases/download/v1.3.1/gnina",
    "https://github.com/gnina/gnina/releases/download/v1.1/gnina",
    "https://github.com/gnina/gnina/releases/download/v1.0.3/gnina",
    "https://github.com/gnina/gnina/releases/download/v1.0.1/gnina",
    "https://github.com/gnina/gnina/releases/download/v1.0/gnina"
]

gnina_working = False

for link in gnina_links:
    print("\nTrying:", link)

    !rm -f gnina
    !wget -q -O gnina "{link}"
    !chmod +x gnina

    if os.path.exists("gnina"):
        size = os.path.getsize("gnina")
        print("Downloaded file size:", size, "bytes")

        if size > 1000000:
            result = subprocess.run(
                ["/gnina", "--help"],
                stdout=subprocess.PIPE,
                stderr=subprocess.PIPE,
                text=True
            )

            output = result.stdout + result.stderr

            if "gnina" in output.lower() or "usage" in output.lower():
                print("GNINA is working.")
                print(output[:1000])
                gnina_working = True
                break
            else:
                print("Downloaded but did not run correctly.")
                print(output[:500])
        else:
            print("File too small. This is probably not the real GNINA binary.")
    else:
        print("Download failed.")

if not gnina_working:
    raise RuntimeError("GNINA did not download correctly from any tested release link.")

```

Trying: <https://github.com/gnina/gnina/releases/download/v1.3.2/gnina>
 Downloaded file size: 0 bytes
 File too small. This is probably not the real GNINA binary.

Trying: <https://github.com/gnina/gnina/releases/download/v1.3.1/gnina>
 Downloaded file size: 0 bytes
 File too small. This is probably not the real GNINA binary.

Trying: <https://github.com/gnina/gnina/releases/download/v1.1/gnina>
 Downloaded file size: 306470832 bytes
 GNINA is working.

Input:

-r [--receptor] arg	rigid part of the receptor
--flex arg	flexible side chains, if any (PDBQT)
-l [--ligand] arg	ligand(s)
--flexres arg	flexible side chains specified by comma separated list of chain:resid
--flexdist_ligand arg	Ligand to use for flexdist

```
--flexdist arg          set all side chains within specified
                        distance to flexdist_ligand to flexible
--flex_limit arg        Hard limit for the number of flexible
                        residues
--flex_max arg          Retain at at most the closest flex_max
                        flexible residues

Search space (required):
--center_x arg          X coordinate of the center
--center_y arg          Y coordinate of the center
--center_z arg          Z
```

```
# Cell 1: Setup environment and project folders
```

```
!pip install biopython requests pandas matplotlib py3Dmol rdkit -q # Install Python libraries
!apt-get update -qq # Update Linux package list
!apt-get install -y openbabel zip wget -qq # Install Open Babel and ZIP tools

import os # For folder/file handling
import io # For reading BLAST XML text
import re # For extracting docking scores
import gzip # For reading compressed GNINA output
import shutil # For creating ZIP file
import requests # For downloading online files
import pandas as pd # For creating result tables
import matplotlib.pyplot as plt # For plotting figures
import py3Dmol # For 3D structure display in Colab
import warnings # To hide unnecessary warnings

from Bio import Entrez, SeqIO # For NCBI sequence retrieval and FASTA/GenBank handling
from Bio.Blast import NCBIWWW, NCBIXML # For online BLAST and parsing BLAST result
from Bio.PDB import PDBParser, PDBIO, Select, Superimposer # For PDB handling and RMSD comparison

warnings.filterwarnings("ignore") # Hide non-critical warnings

project_name = "KRAS_Bioinformatics_Pipeline" # Main project folder name

folders = [
    project_name,
    f"{project_name}/sequences",
    f"{project_name}/structures",
    f"{project_name}/ligands",
    f"{project_name}/blast_results",
    f"{project_name}/alphafold_results",
    f"{project_name}/docking_results",
    f"{project_name}/figures",
    f"{project_name}/tables"
]

for folder in folders:
    os.makedirs(folder, exist_ok=True) # Create folder only if it does not already exist

if os.path.exists("gnina"):
    !chmod +x gnina # Make GNINA executable
    print("GNINA found and ready.")
    !./gnina --help | head -n 5 # Quick test that GNINA is working
else:
    print("GNINA not found. Upload your downloaded GNINA file and name it exactly: gnina")

print("Project setup completed:", project_name)

W: Skipping acquire of configured file 'main/source/Sources' as repository 'https://r2u.stat.illinois.edu/ubuntu jammy InRelease
GNINA found and ready.

Input:
-r [ --receptor ] arg          rigid part of the receptor
--flex arg                    flexible side chains, if any (PDBQT)
-l [ --ligand ] arg            ligand(s)
Project setup completed: KRAS_Bioinformatics_Pipeline
```

```
# Cell 2: Define source links and project IDs
```

```
Entrez.email = "your_email@example.com" # Required by NCBI Entrez; replace with your email

gene_name = "KRAS" # Selected oncogene
organism = "Homo sapiens" # Human KRAS selected for cancer relevance
```

```

ncbi_mrna_id = "NM_004985.5"
ncbi_protein_id = "NP_004976.2"
uniprot_id = "P01116"
pdb_id = "6OIM"
sotorasib_cid = "137278711"

# NCBI RefSeq mRNA accession for KRAS
# NCBI RefSeq protein accession for KRAS
# UniProt accession for human KRAS protein
# Experimental KRAS G12C structure with Sotorasib/AMG 510
# PubChem CID for Sotorasib

links = {
    "NCBI KRAS Gene": "https://www.ncbi.nlm.nih.gov/gene/3845",
    "NCBI KRAS mRNA NM_004985.5": "https://www.ncbi.nlm.nih.gov/nucleotide/NM_004985",
    "NCBI KRAS Protein NP_004976.2": "https://www.ncbi.nlm.nih.gov/protein/NP_004976.2",
    "UniProt KRAS P01116": "https://www.uniprot.org/uniprotkb/P01116/entry",
    "AlphaFold KRAS P01116": "https://alphafold.ebi.ac.uk/entry/P01116",
    "RCSB PDB 6OIM": "https://www.rcsb.org/structure/6OIM",
    "PubChem Sotorasib": "https://pubchem.ncbi.nlm.nih.gov/compound/137278711",
    "NCBI BLAST": "https://blast.ncbi.nlm.nih.gov/Blast.cgi",
    "GNINA": "https://github.com/gnina/gnina"
}

source_links_file = f"{project_name}/source_links.txt"
# File where all source links will be saved

with open(source_links_file, "w") as f:
    for name, link in links.items():
        f.write(f"{name}: {link}\n")
# Save each source link for documentation

print("Project IDs and source links saved.")
print("Saved:", source_links_file)

```

Project IDs and source links saved.
 Saved: KRAS_Bioinformatics_Pipeline/source_links.txt

Cell 3: Retrieve and process KRAS sequences

```

handle = Entrez.efetch(
    db="nucleotide",
    id=ncbi_mrna_id,
    rettype="gb",
    retmode="text"
)
# NCBI nucleotide database
# KRAS mRNA accession
# GenBank format because it contains CDS annotation

record = SeqIO.read(handle, "genbank")
handle.close()
# Read downloaded GenBank record

dna = None
# Empty variable to store coding DNA sequence

for feature in record.features:
    if feature.type == "CDS":
        dna = feature.extract(record.seq)
        break
# CDS = coding sequence region
# Extract only the coding DNA sequence

if dna is None:
    raise ValueError("CDS region not found in KRAS record.")
# Stop if coding region is missing

mrna = dna.transcribe()
protein = mrna.translate(to_stop=True)
# Convert DNA to mRNA by T → U
# Translate mRNA into amino acid sequence until stop codon

dna_file = f"{project_name}/sequences/kras_coding_dna.fasta"
mrna_file = f"{project_name}/sequences/kras_mrna.fasta"
protein_file = f"{project_name}/sequences/kras_translated_protein.fasta"

with open(dna_file, "w") as f:
    f.write(">KRAS_coding_DNA_from_NM_004985.5\n")
    f.write(str(dna) + "\n")
# Save coding DNA FASTA

with open(mrna_file, "w") as f:
    f.write(">KRAS_mRNA_transcribed_from_DNA\n")
    f.write(str(mrna) + "\n")
# Save mRNA FASTA

with open(protein_file, "w") as f:
    f.write(">KRAS_protein_translated_from_mRNA\n")
    f.write(str(protein) + "\n")
# Save translated protein FASTA

handle = Entrez.efetch(
    db="protein",
    id=ncbi_protein_id,
    rettype="fasta",
    retmode="text"
)
# NCBI protein database
# Official KRAS protein accession

```

```
official_protein = SeqIO.read(handle, "fasta") # Read official KRAS protein sequence
handle.close()

official_protein_file = f"{project_name}/sequences/kras_official_protein_NP_004976.2.fasta"
SeqIO.write(official_protein, official_protein_file, "fasta") # Save official NCBI protein FASTA

uniprot_url = f"https://rest.uniprot.org/uniprotkb/{uniprot_id}.fasta"
uniprot_file = f"{project_name}/sequences/kras_uniprot_P01116.fasta"

response = requests.get(uniprot_url) # Download UniProt protein FASTA
response.raise_for_status() # Stop if download fails

with open(uniprot_file, "w") as f:
    f.write(response.text) # Save UniProt FASTA

sequence_summary = pd.DataFrame([
    ["KRAS coding DNA", ncbi_mrna_id, len(dna), dna_file],
    ["KRAS mRNA", "Converted from DNA", len(mrna), mrna_file],
    ["Translated KRAS protein", "Translated from mRNA", len(protein), protein_file],
    ["Official KRAS protein", ncbi_protein_id, len(official_protein.seq), official_protein_file],
    ["UniProt KRAS protein", uniprot_id, "See FASTA", uniprot_file]
], columns=["Sequence", "Accession/Source", "Length", "File"])

sequence_summary_file = f"{project_name}/tables/sequence_summary.csv"
sequence_summary.to_csv(sequence_summary_file, index=False) # Save sequence summary table

print("Sequence retrieval and processing completed.")
print("DNA length:", len(dna), "bp")
print("mRNA length:", len(mrna), "nt")
print("Translated protein length:", len(protein), "aa")
print("KRAS hotspots: G12 =", protein[11], "| G13 =", protein[12], "| Q61 =", protein[60])

sequence_summary
```

Sequence retrieval and processing completed.
DNA length: 567 bp
mRNA length: 567 nt
Translated protein length: 188 aa
KRAS hotspots: G12 = G | G13 = G | Q61 = Q

	Sequence	Accession/Source	Length	File
0	KRAS coding DNA	NM_004985.5	567	KRAS_Bioinformatics_Pipeline/sequences/kras_co...
1	KRAS mRNA	Converted from DNA	567	KRAS_Bioinformatics_Pipeline/sequences/kras_mr...
2	Translated KRAS protein	Translated from mRNA	188	KRAS_Bioinformatics_Pipeline/sequences/kras_tr...
3	Official KRAS protein	NP_004976.2	188	KRAS_Bioinformatics_Pipeline/sequences/kras_of...
4	UniProt KRAS protein	P01116 See FASTA		KRAS_Bioinformatics_Pipeline/sequences/kras_un...

Next steps: [Generate code with sequence_summary](#) [New interactive sheet](#)

```
# Cell 4: Run BLAST and save results

print("Running BLAST. This may take a few minutes...")

blast_result = NCBIWWW.qblast(
    program="blastn", # Nucleotide BLAST
    database="nt", # NCBI nucleotide database
    sequence=str(dna), # KRAS coding DNA sequence
    hitlist_size=5 # Return top 5 hits
)

blast_xml = blast_result.read() # Read BLAST result in XML format

blast_xml_file = f"{project_name}/blast_results/kras_dna_blast_results.xml"

with open(blast_xml_file, "w") as f:
    f.write(blast_xml) # Save raw BLAST XML

blast_record = next(NCBIXML.parse(io.StringIO(blast_xml))) # Parse BLAST XML result

blast_rows = []

for i, hit in enumerate(blast_record.alignments):
```

```

hsp = hit.hsps[0]                                # Take best alignment for each hit
identity = round((hsp.identities / hsp.align_length) * 100, 2) # Calculate percentage identity

blast_rows.append({
    "Rank": i + 1,
    "Hit": hit.title,
    "Identity_percent": identity,
    "E_value": hsp.expect,
    "Alignment_length": hsp.align_length
})

blast_table = pd.DataFrame(blast_rows)             # Convert BLAST result into table

blast_csv = f"{project_name}/blast_results/kras_dna_blast_summary.csv"
blast_table.to_csv(blast_csv, index=False)         # Save BLAST summary as CSV

blast_figure_file = f"{project_name}/figures/blast_summary_table.png"

short_table = blast_table.copy()
short_table["Hit"] = short_table["Hit"].str[:45] + "..." # Shorten long hit names for figure

fig, ax = plt.subplots(figsize=(12, 4))
ax.axis("off")

table = ax.table(
    cellText=short_table.values,
    colLabels=short_table.columns,
    cellLoc="center",
    loc="center"
)

table.auto_set_font_size(False)
table.set_fontsize(8)
table.scale(1, 1.5)

plt.title("KRAS DNA BLAST Summary")
plt.tight_layout()
plt.savefig(blast_figure_file, dpi=200)           # Save BLAST table image
plt.show()

print("BLAST analysis completed.")
print("BLAST XML:", blast_xml_file)
print("BLAST CSV:", blast_csv)
print("BLAST figure:", blast_figure_file)

blast_table

```

Running BLAST. This may take a few minutes...

KRAS DNA BLAST Summary

Rank	Hit	Identity_percent	E_value	Alignment_length
1	gi 2955773857 gb PV477844.1 Homo sapiens	100.0	0.0	567
2	gi 2697689179 ref XM_054664451.2 PREDICTED: P...	100.0	0.0	567
3	gi 2695771275 ref XM_054443149.2 PREDICTED: P...	100.0	0.0	567
4	gi 2697689178 ref XM_528758.9 PREDICTED: Pan ...	100.0	0.0	567
5	gi 2695771274 ref XM_054443148.2 PREDICTED: P...	100.0	0.0	567

BLAST analysis completed.
BLAST XML: KRAS_Bioinformatics_Pipeline/blast_results/kras_dna_blast_results.xml
BLAST CSV: KRAS_Bioinformatics_Pipeline/blast_results/kras_dna_blast_summary.csv
BLAST figure: KRAS_Bioinformatics_Pipeline/figures/blast_summary_table.png

Rank	Hit	Identity_percent	E_value	Alignment_length	
0	1 gi 2955773857 gb PV477844.1 Homo sapiens isol...	100.0	0.0	567	
1	2 gi 2697689179 ref XM_054664451.2 PREDICTED: P...	100.0	0.0	567	
2	3 gi 2695771275 ref XM_054443149.2 PREDICTED: P...	100.0	0.0	567	
3	4 gi 2697689178 ref XM_528758.9 PREDICTED: Pan ...	100.0	0.0	567	
4	5 gi 2695771274 ref XM_054443148.2 PREDICTED: P...	100.0	0.0	567	

Next steps: [Generate code with blast_table](#) [New interactive sheet](#)

```
# Cell 5: Retrieve and prepare protein structures

pdb_url = f"https://files.rcsb.org/download/{pdb_id}.pdb" # Direct PDB download URL
pdb_file = f"{project_name}/structures/kras_g12c_sotorasib_60IM.pdb"

response = requests.get(pdb_url) # Download 60IM experimental structure
response.raise_for_status()

with open(pdb_file, "w") as f: # Save PDB file
    f.write(response.text)

pdb_info_file = f"{project_name}/structures/60IM_pdb_info.txt"

pdb_info_lines = []

with open(pdb_file, "r") as f:
    for line in f:
        if line.startswith(("HEADER", "TITLE", "EXPDTA", "REMARK 2 RESOLUTION")):
            pdb_info_lines.append(line.rstrip()) # Extract structure method and resolution info

with open(pdb_info_file, "w") as f:
    for line in pdb_info_lines:
        f.write(line + "\n") # Save PDB information

het_residues = []

with open(pdb_file, "r") as f:
    for line in f:
        if line.startswith("HETATM"):
            het_residues.append(line[17:20].strip()) # Extract non-protein residues such as ligand, GDP, ions,

het_table = pd.Series(het_residues).value_counts().reset_index()
het_table.columns = ["Residue", "Atom_count"]

het_csv = f"{project_name}/tables/60IM_HETATM_residues.csv"
het_table.to_csv(het_csv, index=False) # Save ligand/non-protein residue table

class KeepProteinOnly(Select):
    def accept_residue(self, residue):
```

```
return residue.id[0] == " " # Keep only standard amino acid residues

parser = PDBParser(QUIET=True)
structure = parser.get_structure("KRAS_60IM", pdb_file)

clean_receptor_file = f"{project_name}/docking_results/kras_60IM_clean_receptor.pdb"

saver = PDBIO()
saver.set_structure(structure)
saver.save(clean_receptor_file, KeepProteinOnly()) # Save cleaned receptor for docking

alphafold_file = f"{project_name}/alphafold_results/kras_alphafold_AF_P01116_F1.pdb"

alphafold_urls = [
    "https://alphafold.ebi.ac.uk/files/AF-P01116-F1-model_v6.pdb",
    "https://alphafold.ebi.ac.uk/files/AF-P01116-F1-model_v5.pdb",
    "https://alphafold.ebi.ac.uk/files/AF-P01116-F1-model_v4.pdb",
    "https://alphafold.ebi.ac.uk/files/AF-P01116-F1-model_v3.pdb",
    "https://models.rcsb.org/AF_AFP01116F1.pdb",
    "https://files.rcsb.org/download/AF_AFP01116F1.pdb"
]

downloaded = False

for url in alphafold_urls:
    print("Trying AlphaFold URL:", url)
    try:
        response = requests.get(url, timeout=30)
        if response.status_code == 200 and "ATOM" in response.text: # Save only if file contains real structure atoms
            with open(alphafold_file, "w") as f:
                f.write(response.text)
            downloaded = True
            print("AlphaFold downloaded from:", url)
            break
    except Exception as e:
        print("Failed:", e)

if not downloaded:
    raise RuntimeError("AlphaFold KRAS structure could not be downloaded.")

print("Structure retrieval completed.")
print("Experimental PDB:", pdb_file)
print("Clean receptor:", clean_receptor_file)
print("AlphaFold PDB:", alphafold_file)

het_table
```

Trying AlphaFold URL: https://alphafold.ebi.ac.uk/files/AF-P01116-F1-model_v6.pdb
AlphaFold downloaded from: https://alphafold.ebi.ac.uk/files/AF-P01116-F1-model_v6.pdb
Structure retrieval completed.
Experimental PDB: KRAS_Bioinformatics_Pipeline/structures/kras_g12c_sotorasib_60IM.pdb
Clean receptor: KRAS_Bioinformatics_Pipeline/docking_results/kras_60IM_clean_receptor.pdb
AlphaFold PDB: KRAS_Bioinformatics_Pipeline/alphafold_results/kras_alphafold_AF_P01116_F1.pdb

	Residue	Atom_count	
0	HOH	207	
1	MOV	41	
2	GDP	28	
3	MG	1	

Next steps: [Generate code with het_table](#) [New interactive sheet](#)

```
from IPython.display import display, HTML # For adding headings and spacing between outputs in Colab
```

```
# Cell 6: Visualize and validate structures with spacing

display(HTML("<h3>Experimental KRAS G12C Structure: PDB 60IM</h3>")) # Add clear heading before structure display

with open(pdb_file, "r") as f:
    experimental_pdb = f.read() # Read experimental PDB structure as text

view = py3Dmol.view(width=850, height=500) # Create 3D viewer
view.addModel(experimental_pdb, "pdb") # Load experimental 60IM structure
view.setStyle({"cartoon": {"color": "spectrum"}}) # Show protein as colored cartoon
```

```

view.setStyle(
    {"hetflag": True},
    {"stick": {"colorscheme": "greenCarbon", "radius": 0.25}}    # Show ligands/hetero atoms as green sticks
)
view.zoomTo()    # Focus view on whole structure
view.show()    # Display structure in Colab

display(HTML("<br><br><hr><br>"))    # Add gap and horizontal line between outputs

display(HTML("<h3>AlphaFold KRAS Predicted Structure</h3>"))    # Add heading before AlphaFold display

with open(alphafold_file, "r") as f:
    alphafold_pdb = f.read()    # Read AlphaFold predicted structure

view = py3Dmol.view(width=850, height=500)    # Create another 3D viewer
view.addModel(alphafold_pdb, "pdb")    # Load AlphaFold structure
view.setStyle({}, {"cartoon": {"color": "spectrum"}})    # Show AlphaFold structure as cartoon
view.zoomTo()    # Focus on structure
view.show()    # Display AlphaFold structure

display(HTML("<br><br><hr><br>"))    # Add spacing before analysis section

# Compare AlphaFold structure with experimental structure using C-alpha atoms

af_structure = parser.get_structure("AlphaFold_KRAS", alphafold_file) # Parse AlphaFold PDB file
exp_structure = parser.get_structure("Experimental_60IM", clean_receptor_file) # Parse cleaned experimental receptor

af_ca = []    # List for AlphaFold C-alpha atoms
exp_ca = []    # List for experimental C-alpha atoms

for model in af_structure:
    for chain in model:
        for residue in chain:
            if "CA" in residue:
                af_ca.append(residue["CA"])    # Store AlphaFold C-alpha atoms

for model in exp_structure:
    for chain in model:
        for residue in chain:
            if "CA" in residue:
                exp_ca.append(residue["CA"])    # Store experimental C-alpha atoms

n = min(len(af_ca), len(exp_ca))    # Use equal number of atoms for fair comparison

sup = Superimposer()    # Create structure superimposition object
sup.set_atoms(exp_ca[:n], af_ca[:n])    # Align AlphaFold atoms onto experimental atoms

rmsd_file = f"{project_name}/alphafold_results/alphafold_vs_60IM_rmsd.txt"

with open(rmsd_file, "w") as f:
    f.write(f"Aligned residues: {n}\n")    # Save number of aligned residues
    f.write(f"RMSD: {sup.rms:.3f} Angstrom\n")    # Save RMSD value

# Extract AlphaFold pLDDT confidence scores

plddt_values = []    # List for AlphaFold confidence scores

for model in af_structure:
    for chain in model:
        for residue in chain:
            if "CA" in residue:
                plddt_values.append(residue["CA"].get_bfactor())    # pLDDT is stored in B-factor column of AlphaFold PDB

plddt_plot_file = f"{project_name}/figures/alphafold_plddt_plot.png"
plddt_text_file = f"{project_name}/alphafold_results/alphafold_plddt_summary.txt"

display(HTML("<h3>AlphaFold pLDDT Confidence Plot</h3>"))    # Heading before confidence plot

plt.figure(figsize=(12, 4))    # Set plot size
plt.plot(range(1, len(plddt_values) + 1), plddt_values)    # Plot pLDDT per residue
plt.axhline(y=90, linestyle="--", label="Very high")    # Reference line for very high confidence
plt.axhline(y=70, linestyle="--", label="Confident")    # Reference line for confident region
plt.axhline(y=50, linestyle="--", label="Low")    # Reference line for low confidence
plt.xlabel("Residue Number")    # X-axis label

```



```

plt.ylabel("pLDDT Score") # Y-axis label
plt.title("KRAS AlphaFold pLDDT Confidence") # Plot title
plt.legend() # Show legend
plt.tight_layout() # Adjust layout
plt.savefig(plddt_plot_file, dpi=200) # Save pLDDT plot as PNG
plt.show() # Display plot in Colab

display(HTML("<br><br><hr><br>")) # Add spacing after plot

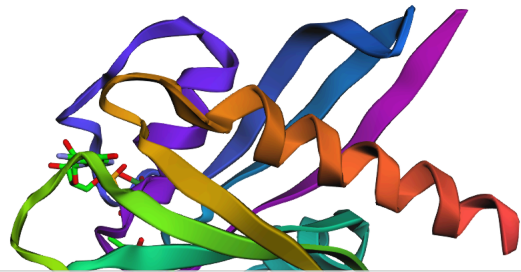
average_plddt = sum(plddt_values) / len(plddt_values) # Calculate average confidence score

with open(plddt_text_file, "w") as f:
    f.write(f"Average pLDDT: {average_plddt:.2f}\n") # Save average confidence
    f.write(f"Number of residues: {len(plddt_values)}\n") # Save number of residues
    f.write(f"Residues above 70: {sum(1 for x in plddt_values if x > 70)}\n") # Save high-confidence residue count

print("Structure validation completed.")
print("Aligned residues:", n)
print("RMSD:", round(sup.rms, 3), "Å")
print("Average pLDDT:", round(average_plddt, 2))
print("RMSD saved:", rmsd_file)
print("pLDDT plot saved:", plddt_plot_file)
print("pLDDT summary saved:", plddt_text_file)

```


Experimental KRAS G12C Structure: PDB 6OIM



```
# Cell 7: Retrieve and prepare Sotorasib ligand

sotorasib_url = f"https://pubchem.ncbi.nlm.nih.gov/rest/pug/compound/cid/{sotorasib_cid}/SDF?record_type=3d"

sotorasib_sdf_file = f"{project_name}/ligands/sotorasib_pubchem_3d.sdf"
sotorasib_pdb_file = f"{project_name}/ligands/sotorasib.pdb"
sotorasib_pdbqt_file = f"{project_name}/ligands/sotorasib.pdbqt"

response = requests.get(sotorasib_url) # Download Sotorasib 3D SDF from PubChem

if response.status_code == 200 and len(response.text) > 100:
    with open(sotorasib_sdf_file, "w") as f:
        f.write(response.text)
else:
    sotorasib_url = f"https://pubchem.ncbi.nlm.nih.gov/rest/pug/compound/cid/{sotorasib_cid}/SDF"
    response = requests.get(sotorasib_url) # Fallback to normal SDF if 3D unavailable
    response.raise_for_status()

    with open(sotorasib_sdf_file, "w") as f:
        f.write(response.text)

!obabel "{sotorasib_sdf_file}" -O "{sotorasib_pdb_file}" --gen3d # Convert ligand to PDB
!obabel "{sotorasib_sdf_file}" -O "{sotorasib_pdbqt_file}" --gen3d # Convert ligand to PDBQT

ligand_summary = pd.DataFrame([
    ["Sotorasib / AMG 510", sotorasib_cid, "PubChem", sotorasib_sdf_file, sotorasib_pdb_file, sotorasib_pdbqt_file]
], columns=["Ligand", "PubChem_CID", "Source", "SDF_file", "PDB_file", "PDBQT_file"])

ligand_summary_file = f"{project_name}/tables/ligand_summary.csv"
ligand_summary.to_csv(ligand_summary_file, index=False) # Save ligand summary table

print("Sotorasib ligand preparation completed.")
ligand_summary
```

1 molecule converted
1 molecule converted
Sotorasib ligand preparation completed.

	Ligand	PubChem_CID	Source	SDF_file	PDB_file
0	Sotorasib / AMG 510	137278711	PubChem	KRAS_Bioinformatics_Pipeline/ligands/sotorasib...	KRAS_Bioinformatics_Pipeline/ligands/sotorasib... KRAS_B

```
# Cell 8: Run GNINA docking

ignored_residues = {"HOH", "WAT", "MG", "NA", "CL", "K", "CA", "GDP", "GTP"} # Ignore water/ions/nucleotides

ligand_candidates = {}

with open(pdb_file, "r") as f:
    for line in f:
        if line.startswith("HETATM"):
            resname = line[17:20].strip()

            if resname not in ignored_residues:
                x = float(line[30:38])
                y = float(line[38:46])
```

```

        z = float(line[46:54])

        ligand_candidates.setdefault(resname, []).append((x, y, z)) # Store possible ligand coordinates

if not ligand_candidates:
    raise ValueError("No ligand found in 60IM for docking box.")

selected_ligand_resname = max(ligand_candidates, key=lambda k: len(ligand_candidates[k])) # Select largest ligand-like residue
coords = ligand_candidates[selected_ligand_resname]

center_x = sum(c[0] for c in coords) / len(coords) # Docking box X center
center_y = sum(c[1] for c in coords) / len(coords) # Docking box Y center
center_z = sum(c[2] for c in coords) / len(coords) # Docking box Z center

box_size = 22 # Docking box size in Å

box_file = f"{project_name}/docking_results/docking_box_values.txt"

with open(box_file, "w") as f:
    f.write(f"Selected ligand residue in 60IM: {selected_ligand_resname}\n")
    f.write(f"center_x = {center_x:.3f}\n")
    f.write(f"center_y = {center_y:.3f}\n")
    f.write(f"center_z = {center_z:.3f}\n")
    f.write(f"box_size = {box_size}\n")

if not os.path.exists("gnina"):
    raise FileNotFoundError("GNINA file not found. Upload GNINA binary and name it: gnina")

!chmod +x gnina

docked_file = f"{project_name}/docking_results/kras_sotorasib_gnina_docked.sdf.gz"
docking_log_file = f"{project_name}/docking_results/gnina_docking_log.txt"

!./gnina \
  -r "{clean_receptor_file}" \
  -l "{sotorasib_sdf_file}" \
  --center_x {center_x} \
  --center_y {center_y} \
  --center_z {center_z} \
  --size_x {box_size} \
  --size_y {box_size} \
  --size_z {box_size} \
  --exhaustiveness 8 \
  --num_modes 10 \
  --seed 42 \
  -o "{docked_file}" \
  2>&1 | tee "{docking_log_file}"

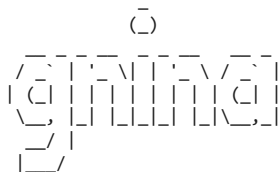
print("GNINA docking completed.")
print("Docking box ligand:", selected_ligand_resname)
print("Docked output:", docked_file)
print("Docking log:", docking_log_file)

```

```

=====
*** Open Babel Warning in PerceiveBondOrders
    Failed to kekulize aromatic bonds in OBMol::PerceiveBondOrders

```



gnina v1.1 master:e4cb380+ Built Dec 18 2023.
gnina is based on smina and AutoDock Vina.
Please cite appropriately.

WARNING: No GPU detected. CNN scoring will be slow.
Recommend running with single model (--cnn crossdock_default2018)
or without cnn scoring (--cnn_scoring=none).

Commandline: ./gnina -r KRAS_Bioinformatics_Pipeline/docking_results/kras_60IM_clean_receptor.pdb -l KRAS_Bioinformatics_Pipeline/kras_sotorasib.sdf --center_x 1.5 --center_y 1.5 --center_z 1.5 --size_x 22 --size_y 22 --size_z 22 --exhaustiveness 8 --num_modes 10 --seed 42 -o docked.sdf.gz 2>&1 | tee docking_log.txt
Using random seed: 42

```

0% 10 20 30 40 50 60 70 80 90 100%
|---|---|---|---|---|---|---|---|---|
*****

```

137278711 | pose 0 | initial pose not within box

mode	affinity (kcal/mol)	intramol (kcal/mol)	CNN pose score	CNN affinity
1	-10.67	1.08	0.3321	7.251
2	-10.01	2.76	0.2542	7.072
3	-6.10	0.19	0.2269	6.483
4	-6.20	1.46	0.1916	6.510
5	-6.48	-0.29	0.1896	6.569
6	-6.78	-1.15	0.1434	6.804
7	-6.70	0.44	0.1415	6.567
8	-9.66	1.04	0.1329	6.896
9	-7.57	0.78	0.1324	6.546
10	-8.10	-1.71	0.1302	6.555

GNINA docking completed.

Docking box ligand: MOV

Docked output: KRAS_Bioinformatics_Pipeline/docking_results/kras_sotorasib_gnina_docked.sdf.gz

Docking log: KRAS_Bioinformatics_Pipeline/docking_results/gnina_docking_log.txt

Check GNINA output files

```
import os
```

```
print("Docked file exists:", os.path.exists(docked_file))
```

```
print("Docked file:", docked_file)
```

```
if os.path.exists(docked_file):
```

```
    print("Docked file size:", os.path.getsize(docked_file), "bytes")
```

```
print("Docking log exists:", os.path.exists(docking_log_file))
```

```
print("Docking log:", docking_log_file)
```

```
if os.path.exists(docking_log_file):
```

```
    print("Docking log size:", os.path.getsize(docking_log_file), "bytes")
```

Docked file exists: True

Docked file: KRAS_Bioinformatics_Pipeline/docking_results/kras_sotorasib_gnina_docked.sdf.gz

Docked file size: 8226 bytes

Docking log exists: True

Docking log: KRAS_Bioinformatics_Pipeline/docking_results/gnina_docking_log.txt

Docking log size: 2011 bytes

Better GNINA tag check

```
import gzip
```

```
import re
```

```
with gzip.open(docked_file, "rt") as f:
```

```
    content = f.read()
```

More flexible tag pattern

```
tags = re.findall(r'>\s*<([^\>]+)>', content)
```

```
print("Number of tags found:", len(tags))
```

```
print("\nUnique tags found:")
```

```
for tag in sorted(set(tags)):
```

```
    print("-", tag)
```

```
print("\nSearching for common score words:")
```

```
for word in ["CNN", "affinity", "Affinity", "score", "Score", "minimized", "Vina", "Vinardo"]:
```

```
    if word in content:
```

```
        print("Found:", word)
```

```
print("\nLast 1000 characters of SDF output:")
```

```
print(content[-1000:])
```

```

19311894 1 18410573972636223624
21033648 29 12606035528402237810
22311459 1 18338799992880404008
244849 19 17131252658708185394
3383291 50 18273495663415402839
3504750 166 18413387653225966582
4409770 3 18342175575181788563
484985 159 18411419522858213299
5080951 261 18339067242523905910
513202 73 18335146439583219083
59755656 520 18200309905532412829
6669772 16 18055636118363107462
6691757 9 18200878378982381024
70251023 43 18049724315018187427

```

```
> <PUBCHEM_SHAPE_MULTIPOLES>
```

```
781.72
```

```
15.2
```

```
6.15
```

```
2.15
```

```
40.49
```

```
4.15
```

```
0.45
```

```
4.84
```

```
-1.73
```

```
-12.84
```

```
1.14
```

```
-2.11
```

```
-1.37
```

```
-0.77
```

```
> <PUBCHEM_SHAPE_SELFOVERLAP>
```

```
1711.255
```

```
> <PUBCHEM_SHAPE_VOLUME>
```

```
423.7
```

```
> <PUBCHEM_COORDINATE_TYPE>
```

```
2
```

```
5
```

```
10
```

```
> <minimizedAffinity>
```

```
-8.10049
```

```
> <CNNscore>
```

```
0.1301983148
```

```
# Cell 9: Analyze, plot, save, and visualize GNINA docking results
```

```

from IPython.display import display, HTML                                # For headings and spacing in Colab
import gzip                                                                # For reading compressed GNINA .sdf.gz output
import re                                                                  # For extracting scores from GNINA SDF output
import pandas as pd                                                        # For creating docking score table
import matplotlib.pyplot as plt                                           # For plotting docking scores
import py3Dmol                                                            # For displaying 3D structures in Colab

display(HTML("<h3>GNINA Docking Score Table</h3>"))                       # Heading for docking score table

# Read GNINA compressed docking output

with gzip.open(docked_file, "rt") as f:                                  # Read complete GNINA SDF output as text
    content = f.read()

# Define GNINA score fields to extract

score_fields = [
    "minimizedAffinity",                                                  # Main docking affinity score; more negative usually means stronger
    "CNNscore",                                                            # CNN pose quality score
    "CNNAffinity",                                                         # CNN-predicted binding affinity
    "CNN_VS",                                                              # Virtual screening score
    "CNNAffinity_variance"                                                # Variance/uncertainty of CNN affinity
]

# Extract scores using flexible SDF tag pattern

scores = {}                                                                # Dictionary to store extracted docking score values

```

```

for field in score_fields:
    pattern = rf'\>s*<{field}>\s*\n(?:\n+)'
    scores[field] = re.findall(pattern, content)

# Flexible pattern works for > <tag> and > <tag>
# Extract all values for each score field

# Check number of docked poses

max_len = max(len(values) for values in scores.values())

# Number of poses detected from score values

if max_len == 0:
    raise ValueError("No GNINA score values found. Check docking output and log file.")

# Create docking score table

rows = []

# Empty list for pose-wise scores

for i in range(max_len):
    rows.append({
        "Pose": i + 1,
        "minimizedAffinity": scores["minimizedAffinity"][i] if i < len(scores["minimizedAffinity"]) else "",
        "CNNscore": scores["CNNscore"][i] if i < len(scores["CNNscore"]) else "",
        "CNNAffinity": scores["CNNAffinity"][i] if i < len(scores["CNNAffinity"]) else "",
        "CNN_VS": scores["CNN_VS"][i] if i < len(scores["CNN_VS"]) else "",
        "CNNAffinity_variance": scores["CNNAffinity_variance"][i] if i < len(scores["CNNAffinity_variance"]) else ""
    })

score_table = pd.DataFrame(rows)

# Convert score rows into dataframe

# Convert score columns to numeric values

for col in score_fields:
    score_table[col] = pd.to_numeric(score_table[col], errors="coerce")

# Save docking score table

score_csv = f"{project_name}/docking_results/gnina_docking_scores.csv"

score_table.to_csv(score_csv, index=False)

# Save docking score table as CSV

display(score_table)

# Display docking score table in Colab

print("Docking score table saved:", score_csv)

display(HTML("<br><br><hr><br>"))

# Add spacing

# Plot minimized affinity scores

display(HTML("<h3>GNINA Docking Affinity Plot</h3>"))

# Heading for docking plot

docking_score_plot_file = f"{project_name}/figures/gnina_docking_scores.png"

plt.figure(figsize=(8, 5))

# Set plot size
plt.bar(score_table["Pose"], score_table["minimizedAffinity"])

# Plot minimized affinity for each pose

plt.axhline(0, linestyle="--")

# Add zero reference line
plt.gca().invert_yaxis()

# Invert y-axis so more negative/stronger binding appears higher

plt.xlabel("Docking Pose")

# X-axis label
plt.ylabel("Minimized Affinity kcal/mol")

# Y-axis label
plt.title("GNINA Docking Scores: KRAS-Sotorasib")

# Plot title
plt.tight_layout()

# Adjust layout
plt.savefig(docking_score_plot_file, dpi=200)

# Save docking score plot
plt.show()

# Display plot

print("Docking score plot saved:", docking_score_plot_file)

display(HTML("<br><br><hr><br>"))

# Add spacing

# Convert best docked pose to PDB format

best_pose_file = f"{project_name}/docking_results/kras_sotorasib_best_pose.pdb"

```

```

!obabel "{docked_file}" -O "{best_pose_file}" -f 1 -l 1          # Convert first/best docked pose from SDF to PDB

print("Best docked ligand pose saved:", best_pose_file)

display(HTML("<br><br><hr><br>"))                                # Add spacing

# Display docked ligand structure only

display(HTML("<h3>Docked Sotorasib Ligand Structure Only</h3>"))

with open(best_pose_file, "r") as f:
    best_pose_pdb = f.read()                                # Read docked ligand PDB file

view = py3Dmol.view(width=650, height=450)                  # Create 3D viewer for ligand only

view.addModel(best_pose_pdb, "pdb")                          # Add docked ligand model
view.setStyle(
    {"model": 0},
    {"stick": {"colorscheme": "magentaCarbon", "radius": 0.25}} # Display docked ligand as magenta sticks
)

view.zoomTo()                                                # Zoom to ligand
view.show()                                                  # Display docked ligand

display(HTML("<br><br><hr><br>"))                                # Add spacing

# Display receptor with best docked Sotorasib pose

display(HTML("<h3>Docked Sotorasib Pose inside KRAS Binding Pocket</h3>"))

with open(clean_receptor_file, "r") as f:
    receptor_pdb = f.read()                                # Read cleaned KRAS receptor PDB

with open(best_pose_file, "r") as f:
    best_pose_pdb = f.read()                                # Read best docked ligand PDB

view = py3Dmol.view(width=850, height=550)                  # Create 3D viewer for receptor + ligand

view.addModel(receptor_pdb, "pdb")                            # Add KRAS receptor model
view.setStyle(
    {"model": 0},
    {"cartoon": {"color": "spectrum"}}                        # Show receptor as cartoon
)

view.addModel(best_pose_pdb, "pdb")                          # Add docked Sotorasib ligand
view.setStyle(
    {"model": 1},
    {"stick": {"colorscheme": "magentaCarbon", "radius": 0.25}} # Show ligand as magenta sticks
)

view.zoomTo({"model": 1})                                    # Zoom to docked ligand
view.show()                                                  # Display receptor-ligand complex

display(HTML("<br><br><hr><br>"))                                # Add spacing

# Save a simple receptor-ligand 3D figure as PNG

display(HTML("<h3>Saved Receptor-Ligand 3D Figure</h3>"))

receptor_coords = []                                         # Store receptor C-alpha coordinates
ligand_coords = []                                           # Store docked ligand atom coordinates

with open(clean_receptor_file) as f:
    for line in f:
        if line.startswith("ATOM") and line[12:16].strip() == "CA":
            x = float(line[30:38])                            # X coordinate
            y = float(line[38:46])                            # Y coordinate
            z = float(line[46:54])                            # Z coordinate
            receptor_coords.append((x, y, z))                 # Save receptor coordinate

with open(best_pose_file) as f:
    for line in f:
        if line.startswith(("HETATM", "ATOM")):
            try:

```



```

        x = float(line[30:38])      # X coordinate
        y = float(line[38:46])      # Y coordinate
        z = float(line[46:54])      # Z coordinate
        ligand_coords.append((x, y, z)) # Save ligand coordinate
    except:
        pass                        # Skip any line that does not contain coordinates

rx = [c[0] for c in receptor_coords] # Receptor X coordinates
ry = [c[1] for c in receptor_coords] # Receptor Y coordinates
rz = [c[2] for c in receptor_coords] # Receptor Z coordinates

lx = [c[0] for c in ligand_coords]   # Ligand X coordinates
ly = [c[1] for c in ligand_coords]   # Ligand Y coordinates
lz = [c[2] for c in ligand_coords]   # Ligand Z coordinates

receptor_ligand_plot_file = f"{project_name}/figures/kras_sotorasib_receptor_ligand_visualization.png"

fig = plt.figure(figsize=(8, 6))      # Create 3D figure
ax = fig.add_subplot(111, projection="3d") # Add 3D axis

ax.plot(rx, ry, rz, label="KRAS receptor") # Plot receptor backbone
ax.scatter(lx, ly, lz, s=30, label="Docked Sotorasib") # Plot docked ligand atoms

ax.set_title("KRAS Receptor with Docked Sotorasib") # Figure title
ax.set_xlabel("X") # X-axis label
ax.set_ylabel("Y") # Y-axis label
ax.set_zlabel("Z") # Z-axis label
ax.legend() # Show legend

plt.tight_layout() # Adjust layout
plt.savefig(receptor_ligand_plot_file, dpi=200) # Save receptor-ligand figure
plt.show() # Display figure

print("Receptor-ligand 3D figure saved:", receptor_ligand_plot_file)

display(HTML("<br><br><hr><br>")) # Add final spacing

# Print final docking summary

best_affinity = score_table["minimizedAffinity"].min() # Most negative value = best predicted binding
best_pose_number = score_table.loc[
    score_table["minimizedAffinity"].idxmin(),
    "Pose"
] # Pose with best/minimum affinity

best_cnnscore = score_table.loc[
    score_table["minimizedAffinity"].idxmin(),
    "CNNScore"
] # CNNScore of best affinity pose

print("Docking results analyzed successfully.")
print("Best pose number:", int(best_pose_number))
print("Best minimizedAffinity:", best_affinity, "kcal/mol")
print("CNNScore of best affinity pose:", best_cnnscore)
print("Docked ligand structure file:", best_pose_file)
print("Docking score CSV:", score_csv)
print("Docking score plot:", docking_score_plot_file)
print("Receptor-ligand figure:", receptor_ligand_plot_file)

```

